



Unidad 4 / Escenario 8

Lectura fundamental

Gestión de la calidad en proyectos de *software*

Contenido

- 1 Métricas y medidas de calidad
- 2 Herramientas de gestión de calidad TSP
- 3 Mejorando la gestión de calidad con datos TSP

Palabras clave:

Defectos, resultados, revisiones, inspecciones.

Introducción

¡Bienvenido nuevamente! En este escenario trataremos la gestión de la calidad en los proyectos TSP. Ya hemos visto la importancia de planear, y cómo utilizar los datos que tenemos para medir y rastrear nuestro progreso. Al rastrear nuestro progreso y los datos de medidas de calidad, podemos identificar problemas antes de que se salgan de nuestro control, y así actuar para corregirlos.

Las metas de calidad normalmente hablan de la cantidad de defectos que se inyectan y se remueven del código para diferentes partes del proyecto. La cantidad de defectos que presenta el *software* son la medida de calidad predominante dentro de TSP.

Hablaremos de las diferentes medidas de calidad y cómo utilizarlas para mejorar la calidad de nuestros proyectos.

1. Métricas y medidas de calidad

En TSP, la calidad del producto se centra en los defectos que tenga cuando fueron inyectados, cuando fueron identificados y arreglados, y cuánto se demoró el equipo en arreglarlos, pero ¿qué es un defecto?

TSP define un defecto como cualquier cosa que esté mal en un programa, al restarle la habilidad a este de cumplir con las necesidades del usuario de forma completa y efectiva; puede ser una falla de ortografía, líneas de código redundantes, secciones que faltan, o una instrucción incorrecta. Los defectos se encuentran en el código, los diseños, requerimientos, o cualquier otra documentación y aparecen en cualquier fase del proceso.

Las siguientes son las medidas de calidad que se utilizan dentro de TSP. La idea es utilizarlas todas en conjunto para obtener una mirada completa de la calidad del producto final:

- Densidad de defectos.
- Tasa de revisiones.
- Proporciones de tiempos de desarrollo.
- Rendimiento.

- Defectos por hora.
- Efectividad de remoción de defectos.
- Proporción de revisión a fallas.

A continuación, las definiremos y hablaremos de todas en detalle.

1.1. Densidad de defectos

Esta métrica mide la cantidad de defectos por la cantidad de LOC nuevas y cambiadas en un programa, expresada en defectos/KLOC (Kilo LOC mil LOC). En otras palabras, un programa con 150 líneas de código y 36 defectos tendría una densidad de defectos de 240 defectos/KLOC.

La densidad de defectos se mide durante todo el proceso de desarrollo. Las pruebas sobre el código no identifican todos los defectos; normalmente quedarán algunos después de las pruebas. Podemos estimar cuántos defectos tendrá nuestro producto con base en cuántos defectos son detectados en las pruebas unitarias.

Vale la pena notar que en los estándares de TSP, 5 o menos defectos/KLOC se considera un programa de buena calidad. El promedio para ingenieros que no han sido entrenados en PSP y TSP es de 20 a 4 defectos /KLOC.

1.2. Tasa de revisiones

Esta se refiere a la velocidad con la cual se revisa el código dentro del equipo. TSP ha encontrado que revisar a una tasa de más de 150 LOC nuevas o modificadas por hora, hace que no se detecten muchos defectos. Idealmente, se debería recopilar información de qué tan rápido se está revisando el código dentro del equipo para determinar cuál es la velocidad aproximada en la cual se detectan la mayoría de los errores.

1.3. Proporciones de tiempos de desarrollo

Estas proporciones tratan de encontrar la relación entre cuánto tiempo se pasa en una fase del proceso a comparación de otra. Hay tres pares de proporciones:

Tiempo en diseño - Tiempo codificando

Esta proporción nos permite determinar la calidad del diseño del ingeniero. Si alguien pasa la mayor parte de su tiempo codificando y no diseñando, es probable que el diseño se esté haciendo a medida que codifica, lo que implica que no estará bien documentado y que quizá sea de mala calidad. Idealmente, un ingeniero debería de pasar al menos igual cantidad de tiempo diseñando su código que codificándolo.

Tiempo en diseño - Tiempo revisando el diseño

A medida que se diseña un componente, se inyectan defectos en él. Sin embargo, al revisar el diseño, casi no se inyectan defectos, pero sí se identifican casi al doble de la proporción que se inyectan al estar diseñando el componente. TSP recomienda que el equipo pase al menos la mitad del tiempo que pasaron diseñando, revisando su diseño para poder identificar y extraer la mayor cantidad de defectos posibles.

Tiempo codificando - Tiempo revisando el código

De forma similar al punto anterior, se inyectan más defectos al producto cuando se está programando que cuando se está revisando el código. El tomarse la molestia de revisar el código de un compañero de equipo mientras que él (o ella) revisa el propio, ayudará a identificar errores rápidamente y a prevenirlos.

1.4. Proporciones de defectos

De forma similar a las proporciones de tiempos de desarrollo, podemos buscar la proporción de defectos encontrados en una parte del proceso con la cantidad encontrada en otra.

Los defectos hallados en revisión de código se comparan con los defectos al compilar, y los defectos hallados al revisar el diseño se comparan con los defectos hallados en las pruebas unitarias. En cualquiera de los dos casos, el equipo debería identificar al menos el doble de defectos durante las fases de revisión que al compilar o al realizar las pruebas unitarias. El no hacerlo puede significar que hay errores en el código que no se han detectado, o que el diseño no está siendo revisado adecuadamente.

La primera proporción nos indica la calidad del código, mientras la segunda nos habla de la calidad del diseño.

1.5. Resultados

Los resultados (*Yield*, en inglés) se calculan midiendo cuántos errores se detectaron en una fase del proyecto y cuántos quedan al terminar de trabajar en ella, esto incluye los defectos que han permanecido de fases anteriores. Para calcular este valor, podemos tomar la cantidad total de defectos inyectados y restarle los defectos que fueron removidos antes de comenzar la fase actual; esto nos dice cuántos de esos defectos nos quedan. Posteriormente, dividimos la cantidad de defectos que eliminamos en la fase actual sobre este valor de defectos restante y esto nos da el *Yield* para una fase particular del proyecto.

La mejor manera de medir la calidad del proceso es el dividir la cantidad de defectos que son eliminados, antes de compilar sobre la cantidad de defectos que fueron inyectados antes de compilar.

1.6. Defectos por hora

Con los datos recopilados, se puede determinar la cantidad de defectos inyectados y eliminados por hora de trabajo, para cada miembro del equipo. Esta medida ayuda a determinar la calidad del código producido individualmente y cuánto tiempo necesitará el individuo para arreglar el código que ha producido.

1.7. Efectividad de remoción de defectos

Aquí medimos la efectividad relativa de dos fases al eliminar defectos y compararlas con una proporción. Su valor está dado al dividir los defectos inyectados por hora sobre los defectos eliminados por hora. Al identificar cuál de todas las fases de remoción de defectos tiene la mejor efectividad al remover defectos, se puede preparar mejor el plan de calidad al concentrar los esfuerzos del equipo en esta fase.

1.8. Proporción de revisión a fallas

Esta última medida consta del costo de las actividades de revisión de código y diseño dividido sobre el costo de las actividades de recuperación de los errores detectados en las pruebas unitarias y la compilación. En otras palabras, el porcentaje de tiempo de revisión de código y de diseño dividido por

el porcentaje de tiempo de compilación y pruebas unitarias, incluyendo en ambos casos el tiempo que se tardan en corregir los defectos hallados en cada fase.

Esta es una buena medida de calidad que también ayuda a identificar cuánto tiempo se dedicó a prevenir los errores durante el desarrollo, contra cuánto tiempo se dedicó a arreglarlos una vez detectados posteriormente.

2. Herramientas de gestión de calidad TSP

Existen varias herramientas que utilizamos en TSP para gestionar la calidad, como medidas históricas de calidad en conjunto con las métricas anteriormente vistas para evaluar la calidad de nuestro código e identificar problemas de forma oportuna.

Las medidas históricas incluyen:

- **Tasas de inyección y eliminación de defectos:** como lo hemos explicado, simplemente la proporción con la cual se inyectan defectos en el código por hora. Dado un estimado de tiempo en una fase del proyecto, estimamos los defectos inyectados y eliminados en esa fase.
- **Costo de calidad (COQ):** determina el esfuerzo empleado para recuperarse de un trabajo defectuoso de un proyecto. Incluye el costo de desempeño, tiempo de revisión, costos de prevención y de falla, (realizar pruebas y corregir los defectos encontrados).
- **Índice de calidad del proceso (PQI):** se miden cinco valores: el diseño, la calidad de las revisiones del diseño, la calidad de las revisiones del código, la calidad del código en sí, y la calidad del programa. Un valor alto indica que el producto no tiene muchos defectos.

Sin embargo, estas medidas no nos dicen nada sin las herramientas adecuadas. Veamos cuales son las herramientas con las que disponemos en TSP:

2.1. Revisiones

Las revisiones se realizan de forma personal, antes que cualquier otra actividad que use el producto y antes que cualquier inspección. El objetivo es buscar y encontrar la mayor cantidad de errores posible y, posteriormente, corregirlos. No solo se deben buscar los errores obvios, sino también los más sutiles.

Se utilizan listas en conjunto con las revisiones. Estas listas incluyen categorías de defectos pasados, y se deben adaptar al proceso de cada individuo, esto con el fin de ayudar a este individuo a encontrar defectos en su propio código, particularmente a los que él o ella es más propenso.

2.2. Inspecciones

Una inspección es una revisión detallada y estructurada, realizada por el dueño (o, en otras palabras, el ingeniero responsable) del producto o componente y otros dos compañeros de trabajo. Puede haber un moderador ayudándolos. Vale la pena notar que las inspecciones como tal no discuten ni intentan solucionar los defectos, solo identificarlos. Las correcciones son realizadas después de revisar el producto.

Se comienza planeando la inspección, con el ingeniero productor y se le entrega el material a revisar al moderador. Luego, en una reunión informativa, el productor revisa el producto y lo explica a los dos evaluadores, y después de esto, los dos moderadores revisan el producto de forma separada para encontrar sus defectos. Una vez se ha revisado el producto, todos se reúnen y revisan el producto una vez más, esta vez juntos. Para finalizar, el productor intenta reparar los defectos y pasa sus correcciones para que sean revisadas.

2.3. Pruebas

Las pruebas corren los programas o sus componentes en un ambiente controlado para verificar que su comportamiento sea el esperado. Las pruebas no deberían utilizarse para encontrar defectos; para esto son las inspecciones y revisiones. Las pruebas deberían validar comportamiento y hacer notar la necesidad de una revisión o inspección.

2.4. Revisiones de calidad

El equipo evalúa los procesos de desarrollo para determinar si se han seguido adecuadamente, con el fin de determinar si se ha hecho todo lo posible por producir un producto de buena calidad.

2.5. Revisiones de los defectos hallados en las pruebas

El equipo evalúa y analiza el producto para determinar cuáles defectos afectaron las pruebas. Se toman las decisiones pertinentes para intentar encontrar y/o prevenir errores similares en el futuro, antes de realizar las pruebas sobre el producto.

3. Mejorando la gestión de calidad con datos TSP

Es importante que los miembros del equipo evalúen los defectos que no identificaron en una de sus revisiones personales e intenten determinar por qué el defecto no fue detectado en el momento. Los datos recopilados en las inspecciones o revisiones de las pruebas permiten identificar aproximadamente en qué fase del proyecto fueron inyectados los defectos y en cuál fueron identificados. Esto a su vez permite repasar el proceso de desarrollo y de revisión del equipo para determinar si hay cambios que se podrían hacer para mejorarlo.

Los valores de los datos obtenidos se deben analizar y reportar en las reuniones semanales cada vez que sea pertinente, particularmente al cambiar de fase en el proyecto y a la hora de realizar las pruebas. Parte del proceso de desarrollo puede incluir la revisión del código producido, tanto por las personales mencionadas anteriormente como una revisión por parte de un compañero de equipo, a modo de prepararse mejor para las inspecciones programadas.

Las estimaciones de tamaño y de tiempos que vimos en escenarios anteriores sirven para estimar las tasas de inyección y eliminación de defectos, sea por LOC o por horas de trabajo. Sea cual sea el caso, se debe de tener mucho cuidado al recopilar información real y acertada sobre ambas tasas, a fin de utilizarla en los relanzamientos del proyecto y en la planeación de proyectos similares.

Los datos de calidad también permiten identificar si el proceso de cada miembro del equipo puede mejorar o no. Al tener control de quién está realizando qué tareas y qué modificaciones han realizado en el código (normalmente, el versionamiento de *software* por medio de git sería suficiente para esto), se puede identificar quién y cuándo inyectó un defecto particular en el código.

El plan de calidad debe ser actualizado con los datos que se obtengan periódicamente, a modo de prevenir más defectos a medida que avanza el proceso de desarrollo.

Referencias bibliográficas

Ampuero, M., & López Trujillo, Y. (2010). *Creando un profesional con disciplina en el proceso de desarrollo de software*. Ingeniería Industrial, 27(1), 4 pág. Recuperado de <http://rii.cujae.edu.cu/index.php/revistaind/article/view/98/77>

Humphrey, W. S. (2000a). *The Personal Software Process SM (PSP SM)*, (November).

Humphrey, W. S. (2000b). *The Team Software Process SM (TSP SM)*, (November).

Humphrey, W. S., Chick, T. A., & Pomeroy-huff, M. (2010). *Team Software Process SM (TSP SM) Body of Knowledge (BOK)*, (July).

Pomeroy-huff, M., & Chick, T. A. (2009). *El Proceso Personal de Software SM (PSP SM)*, 1–36.

INFORMACIÓN TÉCNICA



FACULTAD DE
**INGENIERÍA, DISEÑO
E INNOVACIÓN**

Módulo: Desarrollo de *Software* en Equipo TSP

Unidad 4: Aseguramiento de la calidad

Escenario 8: Gestión de la calidad en proyectos de *software*

Autor: Grissa Maturana

Asesor Pedagógico: Diana Marcela Díaz Salcedo

Diseñador Gráfico: Diego Calderón

Asistente: Ginna Quiroga

Este material pertenece al Politécnico Gran Colombiano. Por ende, es de uso exclusivo de las Instituciones adscritas a la Red Ilumino. Prohibida su reproducción total o parcial.